# UniHub

Security Test Report

App version 0.2.7 (Android and web) | Tested 28-29 October 2026

| **Overall result** | |
|---|---|
| Critical or high findings | **None** |
| Medium findings | 3 (2 fixed, 1 planned before the app store release) |
| Low findings | 4 (all fixed) |
| Database security checks after fixes | **464 passed, 0 failed** |
| Live attack checks against the server after fixes | **44 passed, 0 findings** |
| App tests | 38 passed |
| Known vulnerabilities in third-party packages | 0 of 121 packages |

**NOTE:** Testing was done by the UniHub developer with the help of an AI coding assistant (Claude Code), against the developer's own system only. This is a developer self-assessment, not an audit by an independent security company. All test scripts are included in the project so the results can be repeated by anyone with access (see Appendix A).

# 1. Summary

UniHub is an events platform for universities. Students find and register for events and receive QR tickets; approved organizers publish events; university and platform admins approve organizers and moderate reported events. The app is written in Flutter and uses Supabase (a hosted PostgreSQL database with login, file storage and access rules) as its server.

The test tried to answer one question: **can anyone see or change data they should not?** This was tried as an anonymous visitor, as a new student, and against the database rules for every role.

No user could read another user's registrations, tickets, saved events, reports or verification documents; no one could make themselves an organizer or admin; forged or edited login tokens were rejected; and no secret keys were found in the app or website. Seven weaker points were found. Six are fixed and retested. The remaining one (the Android signing key) only matters once the app is published in an app store and is planned for that step.

# 2. Scope

| In scope | Not in scope (and why) |
|---|---|
| Supabase database: access rules (Row Level Security), functions, permissions, settings | Overload / denial-of-service tests: deliberately not done, to avoid harming the shared Supabase service |
| Supabase login (Auth): passwords, tokens, e-mail codes, account guessing | The web hosting server: UniHub is not hosted yet (planned; see section 6) |
| Supabase file storage: posters, profile photos, private organizer documents | iOS app: not built yet |
| Android app (APK 0.2.6, then 0.2.7): settings, permissions, secrets, signing | Payments (Stripe) and push notifications: not built yet; they will be tested when added |
| Web build: secrets in the code | Supabase's own infrastructure and staff (provider responsibility) |
| Third-party packages: known vulnerabilities | Phishing / social engineering of real users |

# 3. How it was tested

| Phase | What was done |
|---|---|
| Database rule tests | An automated suite (tools/security_tests.py) creates users of every role inside one database transaction, tries allowed and forbidden actions for each, and then undoes everything. It also audits which tables and functions are exposed at all. |
| Anonymous reconnaissance | Using only the public app key (which is visible in every app), tried to read every table, list every storage area and read public settings (tools/pentest/recon.py). |
| Signed-in attacks | Two throw-away accounts were created, then used to attack the live server through the same internet interface the app uses (tools/pentest/api_checks.py): changing own role, reading or changing other users' data, reading hidden data through table links, calling admin functions, forging login tokens, uploading into other folders, disguising a web page as a PDF, guessing which e-mails have accounts. The accounts were deleted afterwards. |
| App analysis | The Android APK was unpacked and its settings, permissions, exported components, signature and contents checked; the web build was searched for secret keys. |
| Dependency audit | All 121 third-party packages were checked against the public OSV vulnerability database. |

| Phase | What was done |
|---|---|
| Settings review | The database was checked for the issues Supabase's own security advisor reports; login settings were reviewed in the Supabase dashboard. |

## Severity levels

| Level | Meaning |
|---|---|
| High / Critical | Another user's private data can be read or changed, or someone can gain admin rights. |
| Medium | A real weakness, but it needs extra conditions or exposes limited information. |
| Low | Hardening: no direct attack found, but the protection is weaker than it should be. |
| Info | Worth knowing; no action needed now. |

# 4. Findings

| ID | Finding | Severity | Status |
|---|---|---|---|
| F1 | Anyone could list all files in the poster and profile-photo storage | Medium | Fixed |
| F2 | The server accepted 6-character passwords (the app requires 8) | Medium | Fixed |
| F3 | The Android app is signed with the shared Android test certificate | Medium | Planned (before store release) |
| F4 | Android backups could copy the login session off the phone | Low | Fixed |
| F5 | The public organizer list revealed roles (who is admin) and universities | Low | Fixed |
| F6 | New database tables and functions were public by default | Low | Fixed |
| F7 | Three database functions had no fixed search path | Low | Fixed |

| F1 Storage areas could be listed | Severity | Status |
|---|---|---|
| | Medium | Fixed |
| **What we found** | Posters and profile photos are stored in public storage so the app can show them by link. The access rule also allowed anyone, even without an account, to *list* every file in those areas. | |
| **Why it matters** | The file list reveals internal user IDs and lets anyone find posters of events that are not published yet. | |
| **What was done** | Listing is now only allowed for a user's own folder (the app needs this to delete a user's files). Viewing a file by its link still works, so the app is unchanged. Private organizer documents were never listable. | |
| **Retest** | Listing all three storage areas returns nothing, anonymously and as a signed-in student (6 checks). Public poster and photo links were confirmed to still load. | |

| F2 Short passwords accepted by the server | Severity | Status |
|---|---|---|
| | Medium | Fixed |
| **What we found** | The app refuses passwords under 8 characters, but the server's own minimum was 6. Someone using the server directly, without the app, could create an account with a 6-character password. | |
| **Why it matters** | Short passwords are much easier to guess. | |
| **What was done** | The minimum password length in the Supabase dashboard was raised to 8, the same rule as the app. Following current NIST guidance (SP 800-63B), length is enforced rather than forced symbol rules. | |
| **Retest** | Signing up directly with a 7-character password is refused by the server ("Password should be at least 8 characters"). This is now part of the automated attack checks. | |

| F3 Test signing certificate | Severity | Status |
|---|---|---|
| | Medium | Planned (before store release) |
| **What we found** | The APK is signed with the standard Android debug certificate, which is the same on every developer's computer. | |
| **Why it matters** | If the app were given to the public this way, anyone could build a fake "update" with the same certificate that Android would accept over the real app. While the app is only installed by the developer and testers, this is not exploitable in practice. | |
| **What was done** | A private release signing key will be created and kept safe before the app is published in an app store (planned as part of the release step). | |
| **Retest** | Will be checked by reading the certificate of the release APK. | |

| F4 Login session included in Android backups | Severity | Status |
|---|---|---|
| | Low | Fixed |
| **What we found** | The app did not switch off Android backups, which are on by default. | |
| **Why it matters** | A phone backup (to the cloud or to a new phone) could carry the stored login session to another device. | |
| **What was done** | Backups and phone-to-phone transfer are now switched off for UniHub (allowBackup=false plus a data extraction rule that excludes all app data). | |
| **Retest** | The settings were confirmed inside the built 0.2.7 APK. | |

| F5 Organizer list revealed roles and universities | Severity | Status |
|---|---|---|
| | Low | Fixed |
| **What we found** | The public list used to show organizer names on events also contained each person's role and university. | |
| **Why it matters** | Anyone could find out which account is the platform administrator, a useful target for an attacker. | |
| **What was done** | The two columns were removed. The list now only has the ID, display name, photo and organization name. | |
| **Retest** | The server refuses requests for those columns; the database suite checks that they cannot return. | |

| F6 New database objects public by default | Severity | Status |
|---|---|---|
| | Low | Fixed |
| **What we found** | The database was set to automatically grant access on every new table, function and sequence to the public and signed-in roles. Every existing object had been locked by hand. | |
| **Why it matters** | One forgotten step when adding a future feature could have exposed new data. | |
| **What was done** | Automatic grants were removed. Every new object must now be opened deliberately. | |
| **Retest** | The database suite checks the default settings and that every exposed object is on an approved list. | |

| F7 Functions without a fixed search path | Severity | Status |
|---|---|---|
| | Low | Fixed |

| | |
|---|---|
| **What we found** | Three internal helper functions did not pin which schema they look up names in. |
| **Why it matters** | Best practice, reported by Supabase's own advisor. No attack path was found, because these functions do not have extra privileges. |
| **What was done** | A fixed (empty) search path was set on all three. |
| **Retest** | The suite now checks that *every* function has a fixed search path, not only privileged ones. |

## Informational

| Topic | Observation |
|---|---|
| Login rate limits | Supabase limits sign-ins and sign-ups to 30 per 5 minutes per internet address, and code checks to 30 per 5 minutes. Eight wrong passwords in a row were therefore not blocked, which is expected. Values are in Appendix B. |
| E-mail code expiry | Set to 10 minutes in the dashboard. Wrong codes are refused. |
| Account guessing | A wrong password and an unknown e-mail give the same answer, so attackers cannot find out who has an account. |
| Error messages / CORS | Server error messages name database tables, and the server accepts requests from any website. Both are normal for Supabase; access is controlled by the rules, not by hiding names. |
| QR scanner package | mobile_scanner is one major version behind (5.2.3 vs 7.x). No known vulnerabilities; upgrade planned. |
| Code shrinking | Android code shrinking (R8) is off. This makes the app easier to read when unpacked, but the app contains no secrets. To be reconsidered for the store release. |

## 5. What was tested and held

A selection of the attacks that were tried and refused by the server, both before and after the fixes:

- A student changing their own role to admin or organizer, their university, or their verified status.
- Reading or renaming another user's profile.
- Reading other users' registrations, tickets, saved events, reports, organizer applications and check-in logs, directly or hidden inside links between tables.
- Seeing events of another university, or other organizers' unpublished drafts.
- A student creating events or calling admin functions (setting roles, listing users, issuing tickets, approving organizers, removing reported events).
- Login tokens with edited contents, with no signature ("alg=none"), or with a broken signature.
- Uploading posters or photos without permission, uploading into another user's folder, and uploading a web page disguised as a PDF.
- Downloading another applicant's private verification documents, or opening them by public link.
- Organizers registering for their own events, reporting events more than 10 times a day, and moderating events of another university.
- Secret keys in the Android app or the website: none found. Only the public app key is present, which is designed to be public; all protection comes from the server rules.

The database suite (464 checks) also covers: ticket check-in rules, capacity limits, reviews only after attending, account deletion removing all personal data, input limits (links, phone numbers, categories), and an audit that every table has access rules switched on.

## 6. Remaining risks and next steps

| Item | Plan |
|---|---|
| Release signing key (F3) | Create a private key before the app store release and keep a backup in a safe place. |
| Website security settings | When the website is hosted: HTTPS only, and browser security headers (blocking framing by other sites, content security policy). Retest after hosting. |
| E-mail sender | Login e-mails currently come from a Gmail account. Before launch, switch to a sender on the UniHub domain with SPF/DKIM, so e-mails are trusted and harder to fake. |
| Password guessing at scale | Rate limits slow down attackers but a distributed attack is still possible. Options for later: CAPTCHA on sign-in, and Supabase's leaked-password check (paid plan). |
| New features | Payments (Stripe) and push notifications will get their own tests and a retest of this suite before release. |
| Keeping it up to date | Rerun the test suites after every database change and before every release; check packages for new vulnerabilities regularly. |

# Appendix A - Repeating the tests

Run from the project folder. The database password is read from a local file that is not part of the code repository; it is never printed.

```
python tools/security_tests.py        # database rules: 464 checks, changes nothing
python tools/pentest/seed.py          # creates 2 throw-away accounts
python tools/pentest/recon.py         # what an anonymous visitor can see
python tools/pentest/api_checks.py    # 44 live attack checks
python tools/pentest/cleanup.py       # deletes the throw-away accounts
```

### Final results (29 October 2026)

| Test | Before fixes | After fixes |
|---|---|---|
| Database rule suite | 447 passed (the new checks did not exist yet) | 464 passed, 0 failed |
| Live attack checks | 37 passed (F1 and F2 were found by the separate storage and password probes) | 44 passed, 0 findings |
| Anonymous storage listing | file lists returned | nothing returned |
| APK backup settings | backup allowed | backup and transfer off |
| App tests | 38 passed | 38 passed |

# Appendix B - Login rate limits (Supabase dashboard, 29 October 2026)

